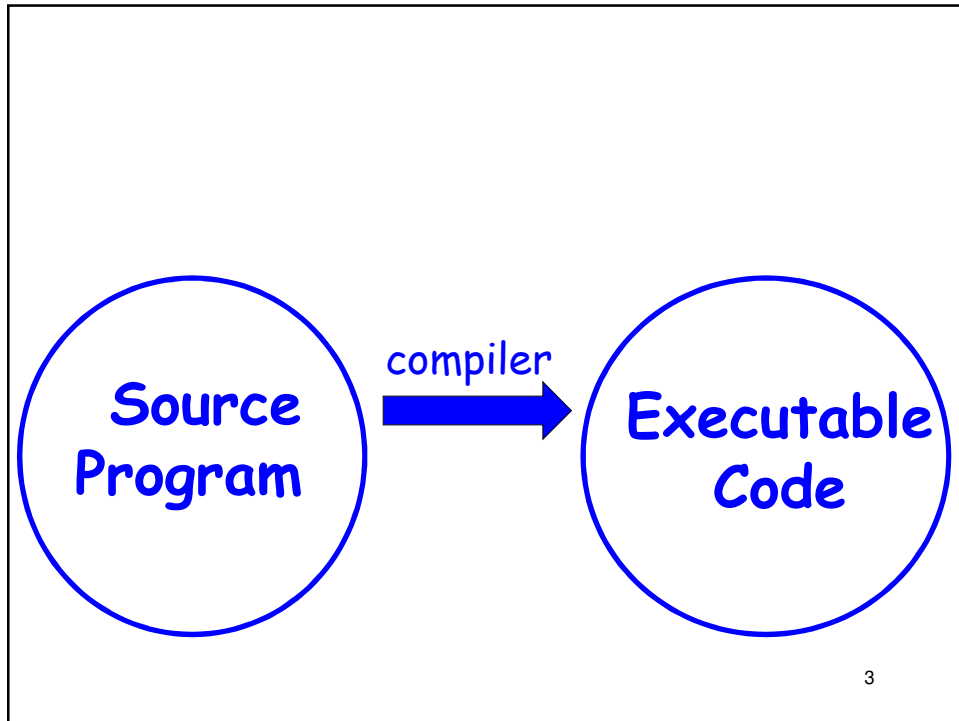


FORMAL LANGUAGES AND COMPILERS

Paola Quaglia

Goals & Expected Learning

- Concepts, techniques, and algorithms for the implementation of compilers
- Formal languages
- Recognition of formal languages



A long way to go

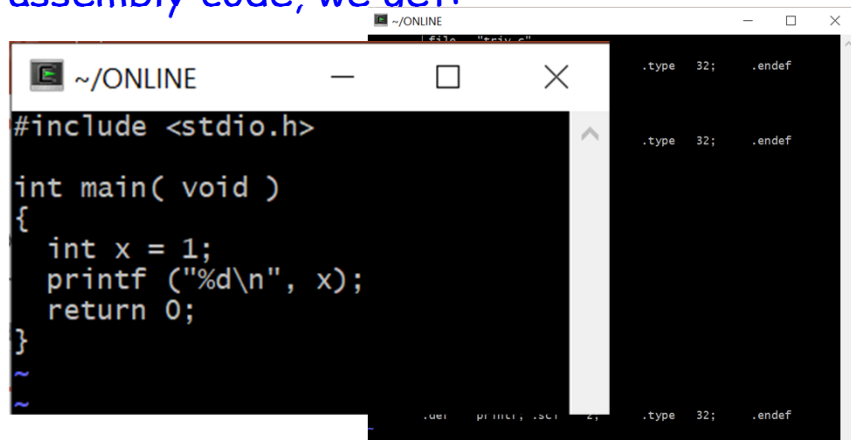
A long way to go

- E.g., the development of gcc has been going on for more than 25 years by now
- Some years ago it already consisted of 2++ million lines of code

5

A long way to go

E.g., from source program to assembly code, we get:



The screenshot shows a code editor window titled '~ / ONLINE'. The left pane displays C source code:

```
#include <stdio.h>

int main( void )
{
    int x = 1;
    printf ("%d\n", x);
    return 0;
}
```

The right pane shows the corresponding assembly code, which includes instructions like `.type 32; .endef` and `.type 32; .endef`.

~ / ONLINE

```
.file "triv.c"
.text
.def __main; .scl 2; .type 32; .endif
.section .rdata,"dr"
.LC0:
.ascii "%d\\12\\0"
.text
.globl main
.def main; .scl 2; .type 32; .endif
.seh_proc main
pushq %rbp
.seh_pushreg %rbp
movq %rsp, %rbp
.seh_setframe %rbp, 0
subq $48, %rsp
.seh_stackalloc 48
.seh_endprologue
call __main
movl $1, -4(%rbp)
movl -4(%rbp), %eax
movl %eax, %edx
leaq .LC0(%rip), %rcx
call printf
movl $0, %eax
addq $48, %rsp
popq %rbp
ret
.seh_endproc
.ident "GCC: (GNU) 9.3.0"
.def printf; .scl 2; .type 32; .endif
```

E.g.,
assembling

~ / ONLINE

```
#include <stdio.h>

int main( void )
{
    int x = 1;
    printf ("%d\\n", x);
    return 0;
}
```

- And assembly code it not yet executable

- Conversion of textual instructions to binary instructions
- Linking to include libraries

Various phases in pipe line

- What we do have:

- The program is written in a language which adheres to a specific grammar
- Some programs are syntactically legal, some are not
- The grammar lets us decide on this

9

Various phases in pipe line

```
pippo = 1;  
pluto = 2;
```

```
while = 1;  
pluto = 2;
```

In most languages a keyword,
cannot be used as an identifier

OK

NOK

10

The grammar

- States the form of an assignment
 - IDENTIFIER ASSIGN EXPRESSION SEMICOLON
- Cannot know "pippo", "pluto", nor the infinitely many strings we may use as variables

11

Lexical analysis

- Translates a stream of characters into a stream of **tokens**
- E.g. translates
- pippo = 2*3;
- Into
- <ID,pippo> ASS <NUM,2> MUL
<NUM,3> SEMCOL
- ID,ASS,NUM,MUL,SEMCOL are tokens

12

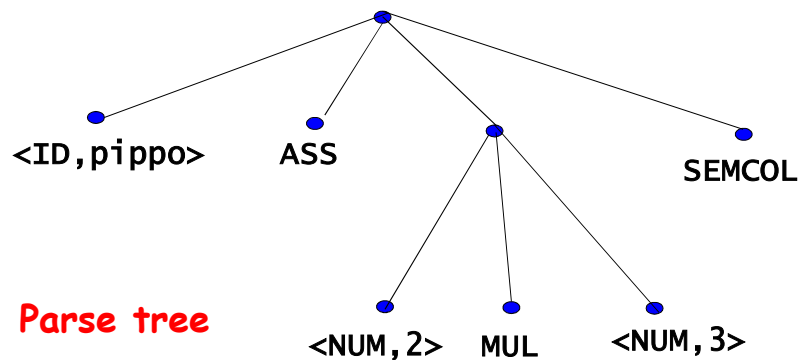
Syntax analysis

- Checks whether a stream of tokens like, e.g.,
 - **<ID,pippo> ASS <NUM,2> MUL <NUM,3> SEMCOL**
- Adheres to the given grammar
- If so, converts the stream of tokens into a **parse tree** (or better, into an **abstract syntax tree**)

13

Syntax analysis

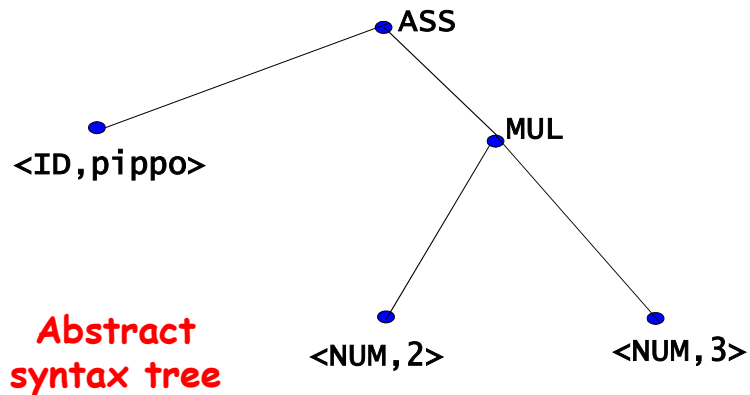
<ID,pippo> ASS <NUM,2> MUL <NUM,3> SEMCOL



14

Syntax analysis

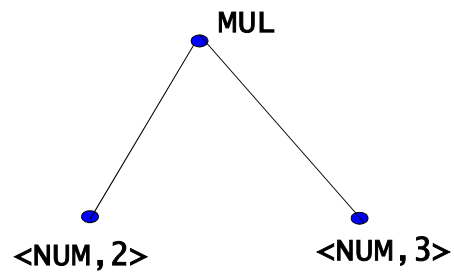
<ID,pippo> ASS <NUM,2> MUL <NUM,3> SEMCOL



15

Semantic analysis

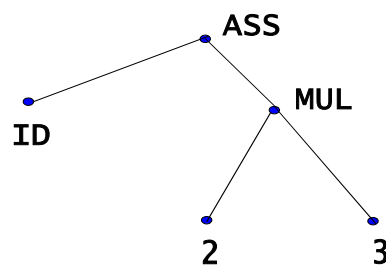
Are we actually multiplying two integers?



16

Intermediate code generation

- Converts a parse tree into a textual intermediate code



```
MOVE 2,R1
MULT 3,R1
MOVE R1,ID
```

17

Target code generation

- Translates intermediate code into target machine code (assembly code)

```
file "triv.c"
.text
.def __main; .sc1 2; .type 32; .endef
.section .data,"dr"
.LC0:
.ascl1 "%d12.0"
.text
.globl main
.def main; .sc1 2; .type 32; .endef
.seh_proc main
main:
pushq %rbp
.seh_pushreg %rbp
movq %rsp, %rbp
.seh_setframe %rbp, 0
subq $48, %rbp
.seh_stackalloc 48
.seh_endprologue
call __main
movl $1, -4(%rbp)
movl -4(%rbp), %eax
movl %eax, %edx
leaq 10(%rip), %rcx
call printf
movl $0, %eax
addq $48, %rbp
popq %rbp
ret
.seh_endproc
.ident "GCC: (GNU) 9.3.0"
.def printf; .sc1 2; .type 32; .endef
```

Plausible targets

- Machine code not necessarily the target of compilation
 - May be a virtual machine
 - May be another language

19

Front-end & Back-end

- Front-end
 - From lexical analysis all the way down to intermediate code generation
- Back-end:
 - All the rest
- Modularity:
 - N languages, N front-ends
 - K machines, K back-ends
 - $N \times K$ compilers, overall

20

That said

21

Goals & Expected Learning

- Pervasive structures (finite state automata, grammars, but also trees and graphs)
- Methodologies at the basis of other areas, too (e.g., parsing of natural languages, soundness of queries to databases)

22

In a Nutshell

- Regular languages and finite state automata (lexical analysis)
- Context-free languages and parsing (syntax analysis)
- Syntax driven definitions (semantic analysis & symbol tables)

23

In a Nutshell (ctd.)

- Intermediate code generation
- Machine code generation
- Optimization techniques

24

Main Textbook

- Compilers: Principles, Techniques, and Tools (2nd edition); Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman; Prentice Hall, 2006

25

Verification

- Written test +
- Oral test

26

Written Test

- Closed books & No electronics

- 5 questions

- Questions on basics (general knowledge of the subject)
- "Mechanical" questions (i.e. application of the algorithms studied in the course to a given structure)
- "Show-off" questions (possibility to show a deeper understanding of the subject)

27

Written Test

- Compulsory subscription on esse3

- In case of subscription & no-show the student undergoes a skip-session

28

Evaluation

- Final score: written and oral contribute 50% each
- *Lode* comes from 30/30 at the written test plus an excellent oral examination

29

Preliminaries: Generative Grammars

30

Generative Grammars Informally

- Fix a vocabulary
 - A set of symbols: **terminals** and **non-terminals**
 - **Terminals**, play the tokens of the output stream of lexical analysis
- E.g. take the vocabulary {S, a, b} where "a" and "b" are terminals

31

Generative Grammars

- One non-terminal symbol of the vocabulary is chosen as **start symbol**
- E.g., S in {S, a, b}

32

Generative Grammars

- Fix a set of **productions**
 - Rules for rewriting strings into strings
 - Constraint:
 - the string to be replaced must contain at least a non-terminal
- E.g. $\{S \rightarrow aSb, S \rightarrow ab\}$

33

Generative Grammars

- These are the ingredients of a **generative grammar**
- A **language** of words of terminals can be generated from the start symbol:
 - Apply the rewriting rules in any possible way, as many times as possible
 - Each rewriting is called a **derivation step**

34

Notation for the derivation relation

$\{S \rightarrow aSb, S \rightarrow ab\}$
 $S \Rightarrow ab$

- Is a one-step derivation from S
- "ab" is made up of terminals only
- Hence "ab" **belongs** to the language generated by the given grammar

35

$\{S \rightarrow aSb, S \rightarrow ab\}$

$S \Rightarrow aSb \Rightarrow aabb$

- Is a two-step derivation from S
- "aabb" is made up of terminals only
- Hence "aabb" **belongs** to the language generated by the given grammar

36

$\{S \rightarrow aSb, S \rightarrow ab\}$

$S \Rightarrow aSb \Rightarrow aaSbb$

- Is a two-step derivation of a string from S
- But "aaSbb" contains a non-terminal
- Hence "aaSbb" **does not belong** to the language generated by the given grammar

37

$\{S \rightarrow aSb, S \rightarrow ab\}$

- Which is the language generated by this grammar?
- $\{a^n b^n \mid n > 0\}$

38

Notation

- Capital letters for non-terminals

39

Convention

- Special character epsilon (ϵ) used to denote the empty word
- Length of ϵ is 0
 - $\epsilon \equiv \epsilon \epsilon$
 - $\epsilon \equiv b^0$ for every terminal b

40

Example 1

$S \rightarrow aAb$

$aA \rightarrow aaAb$

$A \rightarrow \varepsilon$

- Generated language: $\{a^n b^n \mid n > 0\}$
- OBSERVE: Different grammars can generate the same language

41

Example 2

$S \rightarrow AB$

$A \rightarrow aA$

$A \rightarrow a$

$B \rightarrow Bb$

$B \rightarrow b$

- Generated language: $\{a^n b^m \mid n, m > 0\}$

42

Example 3

$S \rightarrow aSBc$

$S \rightarrow abc$

$cB \rightarrow Bc$

$bB \rightarrow bb$

- Generated language: $\{a^n b^n c^n \mid n > 0\}$

43

Example 4

$S \rightarrow AB$

$A \rightarrow a$

- Generated language: $\emptyset$

44

Example 5

$$S \rightarrow \varepsilon$$

- Generated language: $\{\varepsilon\}$
- $\{\varepsilon\} \neq \emptyset$

45

Example 6

$$S \rightarrow aSb$$

$$S \rightarrow \varepsilon$$

- Generated language:
 - $\{a^n b^n \mid n > 0\} \cup \{\varepsilon\} = \{a^n b^n \mid n \geq 0\}$

46

Example

Notation for more productions
with same left-hand side

$S \rightarrow CD$
 $C \rightarrow aCA \mid bCB$
 $AD \rightarrow aD$
 $BD \rightarrow bD$
 $Aa \rightarrow aA$
 $Ab \rightarrow bA$
 $Ba \rightarrow aB$
 $Bb \rightarrow bB$
 $C \rightarrow \varepsilon$
 $D \rightarrow \varepsilon$

- Generated language?

47

Example 7: Derivation 1

$S \rightarrow CD$
 $C \rightarrow aCA \mid bCB$
 $AD \rightarrow aD$
 $BD \rightarrow bD$
 $Aa \rightarrow aA$
 $Ab \rightarrow bA$
 $Ba \rightarrow aB$
 $Bb \rightarrow bB$
 $C \rightarrow \varepsilon$
 $D \rightarrow \varepsilon$

$S \Rightarrow CD$
 $CD \Rightarrow D$
 $D \Rightarrow \varepsilon$

48

Example 7: Derivation 2

$S \rightarrow CD$

$C \rightarrow aCA \mid bCB$

$AD \rightarrow aD$

$BD \rightarrow bD$

$Aa \rightarrow aA$

$Ab \rightarrow bA$

$Ba \rightarrow aB$

$Bb \rightarrow bB$

$C \rightarrow \varepsilon$

$D \rightarrow \varepsilon$

$S \Rightarrow CD$

$CD \Rightarrow aCAD$

$aCAD \Rightarrow aCaD$

$aCaD \Rightarrow aaD$

$aaD \Rightarrow aa$

49

Example 7: Derivation 3

$S \rightarrow CD$

$C \rightarrow aCA \mid bCB$

$AD \rightarrow aD$

$BD \rightarrow bD$

$Aa \rightarrow aA$

$Ab \rightarrow bA$

$Ba \rightarrow aB$

$Bb \rightarrow bB$

$C \rightarrow \varepsilon$

$D \rightarrow \varepsilon$

$S \Rightarrow CD$

$CD \Rightarrow aCAD$

$aCAD \Rightarrow abCBAD$

$abCBAD \Rightarrow abCBA$

$abCBA \Rightarrow abBA$

50

Example 7: Derivation 4

$S \rightarrow CD$

$C \rightarrow aCA \mid bCB$

$AD \rightarrow aD$

$BD \rightarrow bD$

$Aa \rightarrow aA$

$Ab \rightarrow bA$

$Ba \rightarrow aB$

$Bb \rightarrow bB$

$C \rightarrow \varepsilon$

$D \rightarrow \varepsilon$

$S \Rightarrow CD$

$CD \Rightarrow aCAD$

$aCAD \Rightarrow abCBAD$

$abCBAD \Rightarrow abCBaD$

$abCBaD \Rightarrow abCaBD$

$abCaBD \Rightarrow abCabD$

$abCabD \Rightarrow ababD$

$ababD \Rightarrow abab$

51

Generative Grammars Formally

- A grammar is a tuple
 (V, T, S, P)

- V vocabulary of terminals and non-terminals
- T set of terminals
- S start symbol in $(V \setminus T)$
- P set of productions

52

Not

Zero or more repetitions of elements in the base set

- Uppercase, early in the alphabet
 - $A, B, \dots \in (V \setminus T)$
- Uppercase, late in the alphabet
 - $X, Y, \dots \in V$
- Lowercase, early in the alphabet
 - $a, b, \dots \in T$
- Lowercase, early in Greek alphabet
 - $\alpha, \beta, \dots \in V^*$
- Strings of terminals
 - $w, w_0, \dots$

53

Productions

- General form:

$$\delta \rightarrow \beta$$

One or more repetitions of elements in the base

- $\delta \in V^+$
- δ contains at least a non-terminal
- δ called **driver** of the production
- β called **body** of the production

54

Generated Languages

- $G = (V, T, S, P)$

- $L(G) = \{ w \mid w \in T^* \text{ and } S \Rightarrow^* w \}$

T^* because w may just be ϵ

55

Hierarchy of Grammars

- Depending on the shape of productions

- **Context-free grammars**, or just **free grammars**:

- Every production has the form
- $A \rightarrow \beta$
- namely: "non-terminal" arrow "either ϵ or string of terminals and non-terminals"

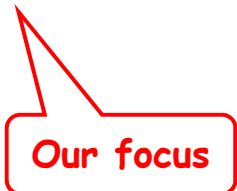
56

Context-free Languages

- L is a context-free language
- Iff
- There exists a context-free grammar G such that $L=L(G)$

57

Context-free Languages



Our focus

58

Canonical Derivations

- **Rightmost (Leftmost) derivation** step:
 - Replace the rightmost (**leftmost**) non-terminal
- **Canonical derivations** of words in the language:
 - Either every derivation step is rightmost
 - Or every derivation step is leftmost

59

Derivation Trees

- Start symbol is the root
- For every derivation step under the production
- $A \rightarrow X_1 X_2 \dots X_n$
- Generate children $X_1 X_2 \dots X_n$ for node A
- Terminals are the leaves (and so is ε)

60

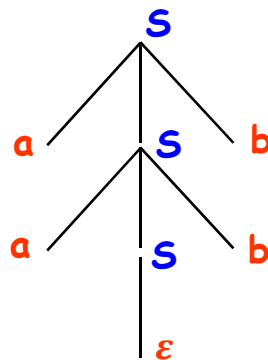
Derivation Trees

- The derived word is at the **frontier** of the tree

61

Example

$$S \rightarrow aSb \mid \varepsilon$$



62

Ambiguity in Natural Languages

- L'uomo guarda la donna con il binocolo

63

Ambiguity

- Grammar G is **ambiguous**
- Iff
- There exists $w \in L(G)$ that can be generated by two distinct canonical derivations, either both rightmost or both leftmost

64

Arithmetic Expressions

$$E \rightarrow E+E \mid E^*E \mid n$$

- Ambiguous?

65

Arithmetic Expressions

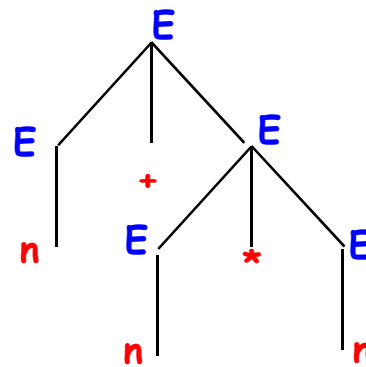
$$E \rightarrow E+E \mid E^*E \mid n$$

- Take $w = n+n*n$

66

Arithmetic Expressions

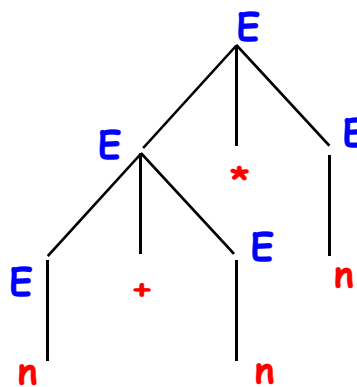
E
 $\Rightarrow E + E$
 $\Rightarrow n + E$
 $\Rightarrow n + E * E$
 $\Rightarrow n + n * E$
 $\Rightarrow n + n * n$



67

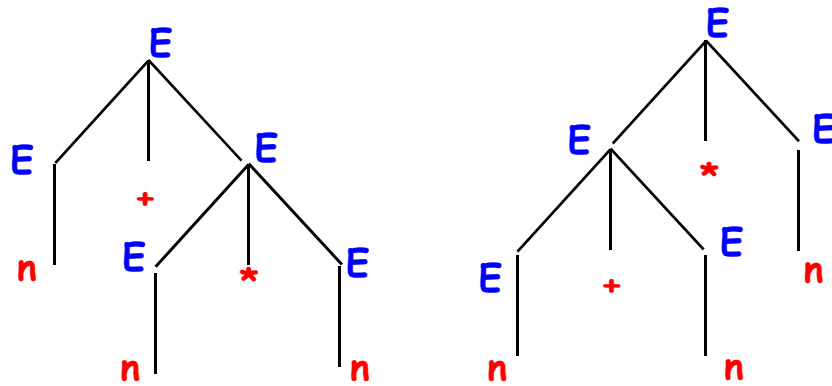
Arithmetic Expressions

But also



68

Arithmetic Expressions



69

Arithmetic Expressions

$$E \rightarrow E + E \mid E * E \mid n$$

- Ambiguous!

70

Dangling Else

$S \rightarrow \text{if } b \text{ then } S \mid \text{if } b \text{ then } S \text{ else } S \mid \text{other}$

- Ambiguous?

71

Dangling Else

$S \rightarrow \text{if } b \text{ then } S \mid \text{if } b \text{ then } S \text{ else } S \mid \text{other}$

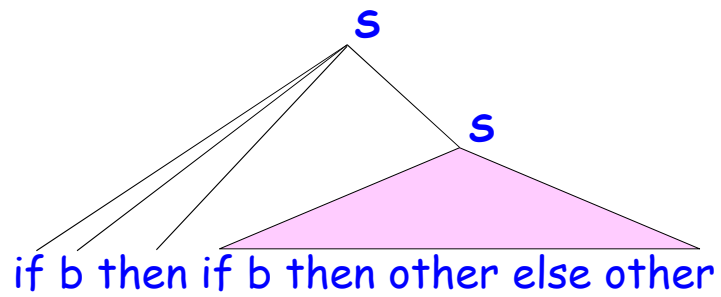
- Take

- $w = \text{if } b \text{ then if } b \text{ then other else other}$

- Which "then" matches "else"?

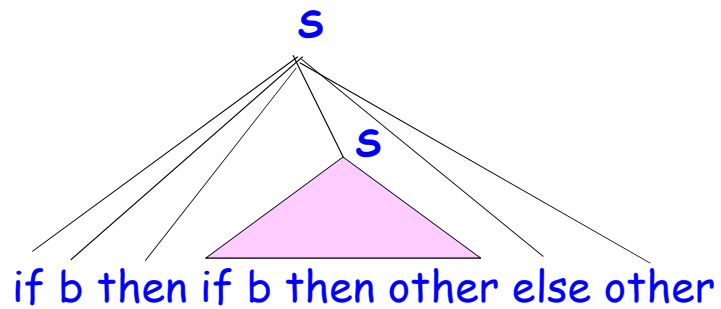
72

Dangling Else



73

Dangling Else



74

Dangling Else

$S \rightarrow \text{if } b \text{ then } S \mid \text{if } b \text{ then } S \text{ else } S \mid \text{other}$

- Ambiguous!

75

Observation

- Ambiguity is undecidable
- No algorithm can be designed to decide whether a grammar is ambiguous or not

76